

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Отчёт по курсу «Современные проблемы прикладной математики»

Математическое моделирование и реализация планировщика процессов для гетерогенных процессорных архитектур

Шаго Павел Евгеньевич

22.Б07-ПУ, 01.03.02 Прикладная математика и информатика

Проверил

кандидат ф.-м.н., доцент

Кривошеин Александр Владимирович

Санкт-Петербург

20 апреля 2026 г.

Постановка задачи

Современные процессорные архитектуры всё чаще объединяют в одном кристалле ядра с принципиально разными характеристиками производительности и энергопотребления — такие платформы принято называть гетерогенными (Intel Hybrid CPU, ARM big.LITTLE).

Стандартный алгоритм планирования Earliest Eligible Virtual Deadline First (EEVDF) ядра Linux не учитывает эту неоднородность и выбирает ядра CPU для выполнения задач вне зависимости от их вычислительной мощности, что может приводить к неоптимальному использованию ресурсов процессора.

Фреймворк `sched_ext` — подсистема ядра Linux, позволяющая реализовывать алгоритмы планирования в виде eBPF программы. Такие программы проходят верификацию и JIT-компиляцию непосредственно при загрузке и исполняются в изолированной виртуальной машине внутри ядра Linux; при этом гарантируется безопасность без ущерба производительности. Ключевое практическое свойство фреймворка состоит в том, что новый планировщик можно загрузить и выгрузить в работающей системе без перекомпиляции и перезагрузки ядра, что существенно сокращает итерационный цикл разработки и экспериментальной проверки улучшенных подходов.

Объектом исследования являются алгоритмы планирования процессов в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой.

Целью выпускной работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом планирования EEVDF ядра Linux.

Обзор литературы

С появлением фреймворка `sched_ext` исследовательская активность в области процессных планировщиков заметно возросла. Большинство современных работ используют `sched_ext` как экспериментальную платформу для быстрой проверки новых алгоритмов без модификации ядра. Диапазон применения охватывает оптимизации существующих алгоритмов, подходы машинного обучения и нестандартные постановки задач.

Tang et al. [2] предложили `sched_ext` планировщик, использующий облегчённую стратегию SRAND с глобальной очередью в eBPF-мап, добившись сокращения времени переключения контекста на 11,83% и роста производительности в стресс-тестах на 7,02% по сравнению с EEVDF. Xu et al. [3] применили `sched_ext` в совершенно иной области — fuzzing-тестировании параллельного кода ядра: специализированный ССТ-планировщик LACE даёт на 38% больше покрытия ветвлений и ускорение исследования в 11,4 раза. В машинном обучении Mao et al. [4] реализовали планировщик Aegis с управлением на основе DeepQ-Network, обеспечивающий нулевую потерю событий аудита там, где EEVDF теряет до 98% записей. Wang et al. [5] показали, что агент на XGBoost, динамически выбирающий политику из портфеля планировщиков, превосходит EEVDF в 86,4% пользовательских сценариев. Galin и Hedblom [6] оценили планировщик LAVD в telco-окружении Kubernetes и обнаружили, что минималистичный `scx_simple` при типичных нагрузках превосходит его, несмотря на значительно меньшую сложность — результат, подчёркивающий ценность `sched_ext` как платформы быстрого сравнения алгоритмов.

На этом фоне задача планирования на гетерогенных процессорах остаётся значительно менее изученной. Наиболее близкой к данному направлению является работа Van Craeynest et al. [7], в которой показано, что интенсивность обращений к памяти — недостаточный критерий распределения задач по типам ядер. Производительность определяется совместным влиянием ILP и MLP, что требует оценки на уровне CPI-компонент. Механизм PIE оценивает производительность на альтернативном типе ядра без фактической миграции и даёт прирост пропускной способности на 5,5–8,7% над конкурирующими подходами. Однако работа относится к 2012 году, выполнена вне контекста `sched_ext` и рассматривает аппаратный механизм оценки, а не программную модель виртуального времени. Именно этот пробел — учёт вычислительной мощности ядра в

рамках программного планирования на современных гетерогенных платформах и составляет предмет выпускной работы.

Промежуточные результаты

Первым этапом реализован планировщик EEVDF [8] на базе `sched_ext`. Каждой задаче при постановке в очередь назначаются виртуальное допустимое время $VE_i = \max(v_i, V_{\text{now}} - \Delta)$ и виртуальный дедлайн $VD_i = VE_i + \Delta S/w_i$, где Δ — квант `sched_ext`, w_i — вес, S — масштаб. Диспетчеризуется задача с наименьшим VD_i .

Сравнение с эталонным EEVDF ядра Linux проводилось на трёх бенчмарках (`hackbench`, `sysbench`, `sched-latency` через `eBPF tracepoints`). По пропускной способности реализация превосходит ядерный планировщик из-за меньшего количества переключений контекста: `hackbench` показал одинаковое время 0.20 с (+0%), `sysbench` вырос с 4157 до 7357 событий/с (+77%), что объясняется глобальной очередью `sched_ext`, сокращающей число миграций задач относительно `per-CPU` очередей ядра. Повышенный 99-й перцентиль задержки (250 мкс против 25 мкс) является ожидаемым следствием `eBPF`-прослойки и не значительно сказывается на быстродействии системы (по `hackbench`).

Вторым этапом реализован прототип планировщика `scx_auction`, основанного на механизме динамического аукциона Викри (VCG). В его основе лежит идея о том, что каждый квант процессорного времени является невозобновляемым ресурсом, а задачи выступают агентами, конкурирующими за этот ресурс в соответствии со своим приоритетом. Планировщик различает P-core и E-core как два класса ресурсов с разной стоимостью кванта ($c_P > c_E$) и разной скоростью исполнения. Для выбора класса ядра используется коэффициент бюджета задачи $\rho_i = B_i/B_i^{\text{max}}$. Задача с высоким ρ_i (мало потребляет, накопила бюджет) расценивается как интерактивная и направляется на P-core, задача с низким ρ_i (CPU-bound) направляется на E-core. Бюджет пополняется пропорционально времени простоя и весу задачи, что создаёт устойчивое равновесие между классами нагрузки. Задачи с исчерпанным бюджетом получают отложенную диспетчеризацию, что приближает механизм к инcentивной совместимости теоретического VCG в условиях бюджетного ограничения.

Список использованных источников

- [1] Tang Q., Gao X., Li G., Lin J. Optimizing Linux Scheduling Based on Global Runqueue with SCX. IEEE International Conference SMC, 2024.
- [2] Xu J., Wolff D., Yi H. X., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF. arXiv, 2025.
- [3] Mao J., Ujcich B. E., Ma S. Rethinking Provenance Completeness with a Learning-Based Linux Scheduler. arXiv, 2025.
- [4] Wang X., Jia S., Huang Z., Cao J., Song M. Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies. arXiv, 2025.
- [5] Galin M., Hedblom A. Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads. Linköping University, 2025.
- [6] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S. Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE). ISCA, ACM, 2012, pp. 213–224.
- [7] Stoica I., Abdel-Wahab H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Old Dominion University, 1996.